Large Language Models
by @jalsbach

1 FOUNDATIONS

1.1 Measure Theory

Probability Space: Triple $(\Omega, \mathcal{F}, \mathbb{P})$:
• $\mathbb{P}$ is a measure • $\mathbb{P}(\Omega) = 1$ • $\mathcal{F} \subseteq 2^{\Omega}$

Ex.: Infinite toin coss paradox:
• $\Omega = \{a,b\}^{\infty}$ • $p(a|\mathbf{y}) = p(b|\mathbf{y}) = \frac{1}{2}$ • $1 = p(\Sigma^{\infty}) = p(\bigcup_{\omega \in \Sigma^{\infty}} \omega) = \sum_{\omega \in \Sigma^{\infty}} p(\omega) = \sum_{\omega \in \Sigma^{\infty}} 0 = 0$ • resolved by setting $p(\text{EOS}|\mathbf{y}) > 0$

σ-Algebra: Set $\mathcal{F} \subseteq \mathcal{P}(\Omega)$ with:
• $\Omega \in \mathcal{F}$ • $\mathcal{E} \in \mathcal{F} \implies \mathcal{E}^C \in \mathcal{F}$ • $\mathcal{E}_1, \mathcal{E}_2, \ldots$ countable sequence of sets in $\mathcal{F} \implies \bigcup_n \mathcal{E}_n \in \mathcal{F}$
$\mathcal{F}$ is σ-Algebra over $\Omega \implies$ tuple $(\Omega, \mathcal{F})$ is a measurable space.

Ex.:
• $\Omega = \{1,2,3\}, \mathcal{F} = \{\emptyset, \{1\}, \{2,3\}, \{1,2,3\}\}$ • $\mathcal{F} = \{\Omega, \emptyset\}$ (trivial σ-algebra) • $\mathcal{F} = \mathcal{P}(\Omega)$ (discrete σ-algebra) • Given $\mathcal{A} \subseteq \Omega, \mathcal{F} := \{\emptyset, \mathcal{A}, \Omega \setminus \mathcal{A}, \Omega\}$ is a σ-algebra induced by $\mathcal{A}$.

Probability Measure: Function $\mathbb{P} : \mathcal{F} \to [0,1]$ over measure space $(\Omega, \mathcal{F})$
• $\mathbb{P}(\Omega) = 1$ • $\mathcal{E}_1, \mathcal{E}_2, \ldots$ countable sequence of disj. sets in $\mathcal{F} \implies \mathbb{P}(\bigcup_n \mathcal{E}_n) = \sum_n \mathbb{P}(\mathcal{E}_n)$

Ex.: $\mathbb{P}([a,b]) = \int_a^b f(x)dx$

Random Variable: Measurable function $X : \Omega \to \mathcal{S}$ for measure spaces $(\Omega, \mathcal{F})$ and $(\mathcal{S}, \mathcal{T})$.

Measurable Function: Function $X : \Omega \to \mathcal{S}$ s.t.: $\epsilon \in \mathcal{T} \implies X^{-1}(\epsilon) \in \mathcal{T}$

1.2 Linear Algebra

• $cos(\theta) = \frac{x \cdot y}{\|x\| \cdot \|y\|}$ • $\|x\| = \sqrt{< x, x >}$

2 LANGUAGE MODELS

2.1 Foundations

Alphabet: Finite, non-empty set Σ. Common compositions:
• $\Sigma^0 = \{\epsilon\}$ • $\Sigma^n = \Sigma \times \cdots \times \Sigma$ • $\Sigma^{\infty} : \Sigma \times \infty \times \Sigma$ • $\overline{\Sigma} = \Sigma \cup \{EOS\}$

String: Finite sequence of symbols drawn from the alphabet.

Ex.: $\Sigma = \{a,b\}$, abaa is a string
Let ϵ be the empty string, $x \cdot \epsilon = \epsilon \cdot x = x$ where $\cdot$ is concatenation. $A \times B = \{x \cdot y | x \in A, y \in B\}$

Kleene Closure: Let Σ be an alphabet. $\Sigma^* = \bigcup_{n=0}^{\infty} \Sigma^n$

Lemma: Σ^* is countable.

Formal Language L (1): Subset of Σ^* for some alphabet Σ.

Formal Language L (2): Probability distribution over Σ^* for alphabet Σ:
• $P(\Sigma^*) = 1$, i.e., $\sum_{x \in \Sigma^*} p(x) = 1$ • $P(\Sigma^{\infty}) = 0$

Sequence Model (SM) 1: Set of conditional probability distributions over Σ: $p_{SM}(x|\mathbf{x})$, $x \in \Sigma, \mathbf{x} \in \Sigma^*$, $\{p(\cdot|\mathbf{x})\}_{\mathbf{x} \in \Sigma^*}$ (countably inf. many)

SM 2: Probability space over $\Sigma^* \cup \Sigma^{\infty}$

Language Model (LM): (Discrete) Probability distr. p_{LM} over Σ^*. Equiv., a LM is a SM s.t. $\mathbb{P}(\Sigma^{\infty}) = 0$

2.2 Globally Normalized LMs

Energy Function: Function $\hat{p} : \Sigma^* \to \mathbb{R}$
Comment: "The lower the better"
How to go from energy function to LM? → (1) make function non-negative and (2) sum to 1.

Globally normalized LM: Let $x \in \Sigma^*, \hat{p}_{GN} : \Sigma^* \to \mathbb{R}$ energy fct.: $p_{LM}(\mathbf{x}) = \frac{e^{\frac{-\hat{p}_{GN}(\mathbf{x})}{T}}}{z}, z = \sum_{x \in \Sigma^*} e^{\frac{-\hat{p}_{GN}(\mathbf{x})}{T}}$
$T \triangleq$ temperature parameter, → lower T makes distr. sharper; higher T makes it flatter/uniform
Comment: problem in practice: z might become $+\infty$; L_{GN} hard to norm. + hard to sample from

Thm.: Any normalizable energy function p_{GN} induces a LM
Comment: $\hat{p}$ is normalizable if $\sum_{\mathbf{y} \in \Sigma^*} e^{-\hat{p}(\mathbf{y})} < \infty$

2.3 Locally normalized LMs

EOS: Distinguished end-of-string symbol: $EOS \notin \Sigma$

Locally normalized LM: Let $\mathbf{x} \in \Sigma^*, |\mathbf{x}| = T$, then: $p(\mathbf{x})_{LN} = p_{SM}(EOS|\mathbf{x}) \cdot \prod_{t=1}^{T} p_{SM}(x_t|\mathbf{x}_{<t})$

2.4 Tightness

Tight LM: Locally norm. LM with: $\sum_{\mathbf{x} \in \Sigma^*} p_{LN}(\mathbf{x}) = 1$

Tight SM: SM with $\mathbb{P}(\Sigma^{\infty}) = 0$ in which case it's also a LM.

Lemma: $\sum_{n=0}^{\infty} x^n = \frac{1}{1-x}$ for $|x| < 1$

Thm.: A LNM is tight i.f.f. $\tilde{p}_{\text{EOS}}(t) = 1$ for some t or $\sum_{t=1}^{\infty} \tilde{p}_{\text{EOS}}(t) = \infty$, with $\tilde{p}_{\text{EOS}}(t) = \frac{\sum_{\omega \in \Sigma^{t-1}} p_{LN}(\omega) p_{LN}(\text{EOS}|\omega)}{\sum_{\omega \in \Sigma^{t-1}} p_{LN}(\omega)}$

Comment: $\sum_{t=0}^{\infty} \frac{1}{ct} = \infty$ for $c \in \mathbb{R}^+$

Prefix probability: Let p be a LM over Σ: $\pi(x) = \sum_{y \in \Sigma^*} p(xy)$

Comment:
• in lecture also denoted as $\overrightarrow{p}(x)$ • π is not a prob. distr. over Σ^* • π is cum. prob. of all strings beginning with x
• $\pi(x|y) = \frac{\pi(yx)}{\pi(y)}$

Thm.: Any LM can be locally normalized: $p_{LM}(\mathbf{y}) = p_{LN}(\mathbf{y}) = p_{SM}(EOS|\mathbf{y}) \prod_{t=1}^{T} p_{SM}(y_t|\mathbf{y}_{<t})$

Proof: Let p be a LM. Define $q(x|\mathbf{x}) \triangleq \overrightarrow{p}(x|\mathbf{x}) = \frac{\overrightarrow{p}(\mathbf{x}x)}{\overrightarrow{p}(\mathbf{x})}$ $q(EOS|\mathbf{x}) \triangleq \frac{p(\mathbf{x})}{\overrightarrow{p}(\mathbf{x})}$
Claim: $q(EOS|\mathbf{x}) \cdot \prod_{t=1}^{T} q(x_t|\mathbf{x}_{<t}) = \frac{p(\mathbf{x})}{\overrightarrow{p}(\mathbf{x})} \cdot \prod_{t=1}^{T} \frac{\overrightarrow{p}(\mathbf{x}_{<t}x_t)}{\overrightarrow{p}(\mathbf{x}_{<t})} = \frac{p(\mathbf{x})}{\overrightarrow{p}(\epsilon)} = p(\mathbf{x})$ (*telescop.series*)
Claim: $\sum x \in \Sigma \cup \{EOS\} q(x|\mathbf{x}) = 1$

Cylinder Set (of rank k): Given set $\mathcal{H} \subseteq \overline{\Sigma}^k$, then: $\overline{C}(\mathcal{H}) := \{y\omega | y \in \mathcal{H}, \omega \in \overline{\Sigma}^{\infty}\}, \overline{C}_k = \{\overline{C}(\mathcal{H}) | \mathcal{H} \in \mathcal{P}(\overline{\Sigma}^k)\}, \overline{C} = \bigcup_{k=1}^{\infty} \overline{C}_k$
→ $\overline{C}(\mathcal{H})$: set of inf. strings that share same k-prefix with some string $\overline{y} \in \mathcal{H}$

Ex.: $\Omega = \{A,B\}$, consider $\mathcal{H} = \Omega^2$, then $C(\mathcal{H}) = \{\omega = \omega_1\omega_2\omega_3... \in \Omega^{\infty} | \omega_1, \omega_2 \in \Omega^2\}$

Thm.: $\mathbb{P}(\overline{C}(\mathcal{H})) = \Sigma_{\overline{y} \in \mathcal{H}} \overrightarrow{p}(\overline{y})$

Lemma: Σ^{∞} is measurable in $(\Sigma^* \cup \Sigma^{\infty}, \sigma(C))$

Comment:
• Non-tight SMs involve setting $p(EOS|x) = 0$ • $p(EOS|x)$ can → 0 and still be tight! Ex.: $p(a|x) = 1 - \frac{1}{|x|+1}$

2.5 Representation-based LMs

Inner product space: Vector space $\mathbb{V}$ over field $\mathcal{F}$ coupled with an inner product map $\langle \cdot, \cdot \rangle : \mathbb{V} \times \mathbb{V} \to F$:
i **Conjugate symmetry**: $\langle v,u \rangle = \overline{\langle u,v \rangle}$ $\forall v,u \in \mathbb{V}$
ii **Linearity in 1st arg.**: $\langle av+bu, z \rangle = a\langle v,z \rangle + b\langle u,z \rangle$ $\forall v,u,z \in \mathbb{V}, a,b \in F$ iii **Positive-definiteness**: $\langle v,v \rangle > 0$ $\forall v \neq 0$

Hilbert space: Inner product space that is **complete** i.e. every Cauchy seq. in $\mathbb{V}$ remains in $\mathbb{V}$:

$\forall$series $\sum_{n=1}^{\infty} \mathbf{v}_n$ s.t. $\sum_{n=1}^{\infty} \|\mathbf{v}_n\| < \infty$, it holds: $\sum_{n=1}^{\infty} \mathbf{v}_n \in \mathbb{V}$

Softmax: $\text{softmax}(x)_j = \frac{e^{x_j}}{\sum_{i=1}^{D} e^{x_i}}, x \in \mathbb{R}^D$

Comment: $\text{softmax}(\frac{x}{T})$ has annealing param. T (temp.); $\lim_{T \to \infty} = uniform$ → most entropic; $\text{softmax}(x+c) = \text{softmax}(x)$ (invariant); derivative: $\frac{\partial \text{softmax}(\mathbf{x})_i}{\partial x_j} = \delta_{ij} \frac{e^{x_i} \sum_k e^{x_k} - e^{x_i} e^{x_j}}{(\sum_k e^{x_k})^2}$
Step 1: Define a SM; **Step 2**: $\{p(\cdot|x)\}_{x \in \Sigma^*}$; **Step 3**: $p(\cdot|x) = \text{softmax}(E \cdot enc(x))$, $enc : \Sigma^* \to \mathbb{R}^D$, $E \in \mathbb{R}^{|\overline{\Sigma}| \times D}$

2.6 N-gram Models

Finite-state Automaton: 5-Tuple $(\Sigma, Q, I, F, \delta)$; Σ: alphabet, Q: finite set of states, $I \subseteq Q$: init. states, $F \subseteq Q$: final states, $\delta \subseteq Q \times \Sigma \cup \{\epsilon\} \times Q$: transitions

Regular language: Language recognized by unweighted FSA

n-gram Model: $\{p(\cdot|\mathbf{x})\}_{\mathbf{x} \in \Sigma^*}$ for which: $p_{SM}(y_t|\mathbf{y}_{<t}) = p_{SM}(y_t|\mathbf{y}_{t-n+1:t-1})$

Comment: n-grams LMs are determ. FS LMs

Comment: #params of n-gram LM: $(|\overline{\Sigma}|-1)\Sigma_{j=0}^{n-1}|\Sigma|^j$

Comment: disadvantages: high params count (needs params sharing), poor perf. with limit. data, no long-range depend.

MLE of n-gram LM: $\theta \overset{\text{def}}{=} \left\{\theta_{y|\mathbf{y}} \overset{\text{def}}{=} p_{\text{SM}}(y|\mathbf{y}) \,|\, y \in \overline{\Sigma}, \mathbf{y} \in \overline{\Sigma}^{n-1}, \theta_{y|\mathbf{y}} \geq 0, \sum_{y' \in \overline{\Sigma}} \theta_{y'|\mathbf{y}} = 1\right\}$
MLE solution: $p_{\text{SM}}(y|\mathbf{y}_{<n}) = \frac{C(y_1,\ldots,y_n)}{C(y_1,\ldots,y_{n-1})}$, C: # occurences

Lidstone smoothing: Add smoothing param. λ. ($\lambda = 1$: Laplace smoothing): $p_{\text{SM}}(y|\mathbf{y}_{<n}) = \frac{C(y_1,\ldots,y_n)+\lambda}{C(y_1,\ldots,y_{n-1})+V\lambda}$, V: vocab. size

Neural Probabilistic LM (NPLM): Concatenates $n-1$ prefix embeddings and passes the result through a feed-forward layer: $y = b + Wx + U\tanh(d + Hx)$
→ can understand similarities betw. words; params shared across different contexts (→ fewer params than n-gram model); trained with GD

3 NEURAL NETWORK MODELING

3.1 RNNs

Recurrent neural sequence model (RNSM): 5-Tuple $(\Sigma, D, \mathbf{f}, \mathbf{E}, \mathbf{h}_0)$ with $\mathbf{E} \in \mathbb{R}^{|\overline{\Sigma}| \times D}$, $f_{\Delta^{|\overline{\Sigma}|-1}} : \mathbb{R}^{|\mathcal{H}|} \to \Delta^{|\overline{\Sigma}|-1}$: $p_SM(y_t|\mathbf{y}_{<t}) = \mathbf{f}_{\Delta^{|\overline{\Sigma}|-1}}(\mathbf{E}\ \text{enc}_{\mathcal{R}}(\mathbf{y}_{<t}))_{y_t} = \mathbf{f}_{\Delta^{|\overline{\Sigma}|-1}}(\mathbf{E}\mathbf{h}_{t-1})_{y_t}$

Thm.: Softmax RNSM with bounded act. fct. f is tight

Linear RNN: • $h_0 \in \mathbb{R}^D$ • $h_t = A(h_{t-1} + Becx_t)$, $h_t \in \mathbb{R}^D$, $A \in \mathbb{R}^{D \times D}, B \in \mathbb{R}^{D \times k}, e : \overline{\Sigma} \to \mathbb{R}^k$

Elman RNN: $h_t = \sigma(Uh_{t-1} + Ve'(t_t) + b_h)$, $e' : \Sigma \to \mathbb{R}^R$ embedding fct., $\mathbf{b}_h \in \mathbb{R}^D, \mathbf{U} \in \mathbb{R}^{D \times D}, \mathbf{V} \in \mathbb{R}^{D \times R}$

Heaviside function: $H(x) = x > 0\ ?\ 1 : 0$

Comment: HE RNNs can only repr. determ. finite-state languages; if σ is replaced with fct. mapping to infinite set it becomes turing complete
PFSA → HE RNN: must be determinizable; $D = |\Sigma| \cdot |Q|$; $R = |\Sigma|$

Ex.: Given $\mathbf{h}_t = f(\mathbf{h}_t, y_t)$; $p(y_t|\mathbf{y}_{<t}) = \text{softmax}(\mathbf{E}\mathbf{h}_{t-1})_{y_t}$: $O(kt + |\overline{\Sigma}|D)$, $p(\mathbf{y})$: $O(|\mathbf{y}||\overline{\Sigma}|D)$ (as this type of RNN induces LN LM)

3.2 Transformer-based LMs

Transformer SM: 4-tuple $(\Sigma, D, enc_{\mathcal{T}}, \mathbf{E})$ defining the SM $p_{SM}(\overline{y}_t|\mathbf{y}_{<t}) = \text{softmax}(\mathbf{E}\mathbf{h}_{t-1})_{\overline{y}t}$, $\mathbf{h}_{t-1} = \text{enc}_{\mathcal{T}}(\mathbf{y}_{<t})$

Attention: Let $f : \mathbb{R}^D \times \mathbb{R}^D \to \mathbb{R}$ scoring fct., $f_{\Delta^{D-1}}$ proj. fct., $\mathbf{q} \in \mathbb{R}^D, \mathbf{K}_t = (\mathbf{k}_1^T, \cdots \mathbf{k}_t^T) \in \mathbb{R}^{t \times D}, \mathbf{V}_t \in \mathbb{R}^{t \times D}$
Attention over $\mathbf{K}_t, \mathbf{V}_t$, denoted by $\text{Att}(q, \mathbf{K}_t, \mathbf{V}_t)$, involves:
$\mathbf{s}_t = \mathbf{f}_{\Delta^{D-1}}\big(f(\mathbf{q}, \mathbf{k}_1), \ldots, f(\mathbf{g}, \mathbf{k}_t)\big)$
$\mathbf{a}_t = (\text{Att}(\mathbf{q}, \mathbf{K}_t, \mathbf{V}_t) = s_1\mathbf{v}_1 + \cdots + s_t\mathbf{v}_t$
Most common choice for f: $f(\mathbf{q}, \mathbf{k}) = \frac{1}{\sqrt{D}} < \mathbf{q}, \mathbf{k} >$

Soft attention: $\mathbf{f}_{\Delta^{D-1}} = \text{softmax}$

Averaging hard attention: $\mathbf{f}_{\Delta^{D-1}} = \text{hardmax}_{\text{avg}}(\mathbf{x})_d = \begin{cases} \frac{1}{|\text{argmax}(\mathbf{x})|} & \text{if } d \in \text{argmax}(\mathbf{x}) \\ 0 & \text{else} \end{cases}$

Unique hard attention: For $\mathbf{x} \in \mathbb{R}^D$, sample $\hat{d} \sim \text{Unif}(\text{argmax}(\mathbf{x}))$ or choose some $\hat{d} \in \text{argmax}(\mathbf{x})$ determ. $\mathbf{f}_{\Delta^{D-1}} = \text{hardmax}_{\text{uni}}(\mathbf{x})_d = \begin{cases} 1 & \text{if } d = \hat{d} \\ 0 & \text{else} \end{cases}$

Transformer layer: Let $Q, K, V, O : \mathbb{R}^D \to \mathbb{R}^D$ and $W_Q, W_K, W_V, W_O \in \mathbb{R}^{D \times D}$ parametrized.
Transformer layer $\mathbf{T} : \mathbb{R}^{T \times D} \to \mathbb{R}^{T \times D}$ for input $\mathbf{X} = (\mathbf{x}_1^T, \cdots \mathbf{x}_T^T)$:

$Q(\mathbf{x}_t) = \mathbf{x}_t\mathbf{W}_Q, K(\mathbf{x}_t) = \mathbf{x}_t\mathbf{W}_K, V(\mathbf{x}_t) = \mathbf{x}_t\mathbf{W}_V, \mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{D \times D}$

$\mathbf{a}_t = \text{Att}(\mathbf{x}_t\mathbf{W}_Q, \mathbf{X}_t\mathbf{W}_K, \mathbf{X}_t\mathbf{W}_V) + \mathbf{x}_t$
$\mathbf{z}_t = O(\mathbf{a}_t) + \mathbf{a}_t$
$\to T(\mathbf{X}) = \mathbf{Z} = (\mathbf{z}_1^T, \cdots \mathbf{z}_T^T) \in \mathbb{R}^{T \times D}$

Transformer: L-layer D-dimensional model over Σ with hidden state $\mathbf{h}_t =$

$\text{enc}_{\mathcal{T}}(y_1...y_t) = \text{enc}_{\mathcal{T}}(\mathbf{y}), 1 \le \ell < L$ and $F : \mathbb{R}^D \to \mathbb{R}^D$:
$\mathbf{X}^1 = \left(\mathbf{e}'(y_0),\ldots,\mathbf{e}'(y_t)\right)$, $\mathbf{Z}^\ell = \mathbf{T}_\ell(\mathbf{X}^\ell)$,
$\mathbf{X}^{\ell+1} = \mathbf{Z}^\ell$, $\mathbf{h}_t = \mathbf{F}(\mathbf{z}_t^\mathbf{L})$ (F: MLP)

Attention Block: Let $\mathbf{X} \in \mathbb{R}^{T\times D}$ input encodings
Attention block $\mathbf{A} : \mathbb{R}^{T\times D} \to \mathbb{R}^{T\times D}$:
$\mathbf{A}(\mathbf{X}) = \mathbf{f}_{\Delta^{D-1}}((\mathbf{X}\mathbf{W}_Q)(\mathbf{X}\mathbf{W}_K)^T)(\mathbf{X}\mathbf{W}_V)$
with **attention matrix** $\mathbf{U} = (\mathbf{X}\mathbf{W}_Q)(\mathbf{X}\mathbf{W}_K)^T \in \mathbb{R}^{T\times T}$
COMPLEXITY
Runtime: • Queries, Keys, Values (e.g. $Q = XW_Q$): $O(TD^2)$ • Attention Matrix: $O(T^2D)$ • Softmax: $O(T^2)$ • Attention block: $O(T^2D)$ • Final proj.: $O(TD^2)$
Params: • each attention layer: $4 \cdot (D \cdot h \cdot H) = 4D^2$ • final MLP (2 lin. layers with hidden dim $4D$): $2 \cdot (D \cdot 4D) = 8D^2$ • total (given L transformer layers): $12D^2L$
Masked Attention Block: $\mathbf{f}_{\Delta^{D-1}}((\mathbf{X}\mathbf{W}_Q)(\mathbf{X}\mathbf{W}_K)^T \odot \mathbf{M})(\mathbf{X}\mathbf{W}_V)$
with $M_{i,j} = \begin{cases} 1 & \text{if } i \le j \\ -\infty & \text{else} \end{cases}$, $0 \le i,j < T$

Positional encoding: Let $\mathbf{e}' : \overline{\Sigma} \to \mathbb{R}^D$ symbol repr.fct. and $\mathbf{f}_{pos} : \mathbb{N} \to \mathbb{R}^D$ positional encoding.
Position-augmented repr. fct. of symbol y_t in string $\mathbf{y}$:
$\mathbf{e}_{pos}(y_t) = \mathbf{e}'(y_t) + \mathbf{f}_{pos}(t)$
Ex.: $n \to [\sin(\frac{n}{2\pi}), \cos(\frac{n}{2\pi})]$; add info about rel./abs. pos. in seq.
Multi-Head Attention Block: Let H be number of attention heads: MH-A$(\mathbf{X}) = \mathbf{f}_\mathbf{H}((\text{concat}_{0\le h<H}((\text{softmax}($
$(Q_h(\mathbf{X})K_h(\mathbf{X})^\top))V_h(\mathbf{X}))))$
head dimension usually: $h = \frac{D}{H}$

4 TRAINING & FINETUNING

4.1 Transfer learning

→ train NN on vast text corpora; then finetune on specific task
Context. Word Vectors (COVE): 1. trained seq2seq for machine translation; 2. used 2-layer bidirec. LSTM encoder as starting point for other NLP tasks e.g. sentiment analysis; 3. concat GloVe embeds of a word with corresp. CoVe vec
ELMO: 1. train separate left-to-right & right-to-left LMs; 2. apply as "pretrained embedds", produces L+1 interm. repres.; ELMo repr. for token y_t: $\gamma^{\text{task}} \sum_{l=0}^{L} s_l^{\text{task}} h_{tl}^{LM}$, where $h_{tl}^{LM} = (\overrightarrow{h}_{tl}^{LM}, \overleftarrow{h}_{tl}^{LM})$ are hidden states of the LM layers; concat ELMO vector with input x_k at lowest layer
BERT: bidir. transformer encoder; Tokenizer: score $= \frac{\text{freq}_{\text{pair}}}{\text{freq}_{\text{first}} \cdot \text{freq}_{\text{second}}}$
RoBERTa: robustly opt. BERT; same model arch.; larger pre-training corpus, batch size input seq. length
ALBERT: A Lite Bert; cross-layer param. sharing, embed matrix decomposed into 2 projecting words to lower dim. embed space to be projected to hidden space; sentence order pred. (SOP) builds pos. examples (same as BERT) and neg. examples by swapping order ; param count of BERT base 108M → 12M
ELECTRA: BERT modif.; pre-trained with Replaced Token Detection (RTP) obj. (self-supervised) instead of generative; generator creates corrupted versions of input data; discriminator trained to dist. between orig. & corr. data at token level
T5: trained with text infilling obj.; model trained on predicting randomly masked text spans

4.2 Parameter-efficient finetuning

IDEA 1: Finetune random small part of network
Specification-based tuning: finetune only a few params; 1/4 of final layers of BERT produce 90% of full-finetuning perf.
BitFit: optimize bias terms in model (both attention & MLP layers) while freezing other params; 95% of full-finetuning perf.
IDEA 2: Learn spec. part of network to be updated
Diff pruning: reparam. finetuned params as sum of pretrained params & sparse finetuning update vector: $\theta_{FT} = \theta_{LM} + \delta_{Diff}$
IDEA 3: Adapters: Add adapter layers within transformer layer; freeze all params & only finetune adapter params; place 2-layer FFN with bottleneck after each sublayer: $\mathbf{h} \leftarrow \mathbf{h} + f(\mathbf{h}\mathbf{W}_{down})\mathbf{W}_{up}$
IDEA 4: Prefix Tuning: freeze transformer params & optimize (acrtivations corresp. to) prefix for each task e.g. summarization, translation; only need to store prefix representations for each task; modular & space-efficient; prefix activations for all layers are learnt; reparam. param matrix P_θ by smaller matrix P'_θ: $P_\theta = MLP_\theta(P'_\theta)$; after training only P_θ saved
IDEA 5: Low Rank Adapt. (LoRA): pretrained matrix $W_0 \in \mathbb{R}^{D\times D}$; finetuning update: $W_0 + \Delta W = W_0 + BA$ with $B \in \mathbb{R}^{d\times r}, A \in \mathbb{R}^{r\times k}$; $rank(r) << min(d,k)$; forward pass with LoRA: $h = W_0x$ (frozen) $+BAx$ (trainable); + doesn't add depth unlike adapters; + easier to optimize than prefix tuning

5 APPLICATIONS & SCALING

5.1 Prompting & Zero-shot inference

+ doesn't require param updates Head-based finetuning: [CLS] + input → output label
Prompting: [CLS] + input + template text + [MASK] → $\mathcal{M}(y)$ (label-word mapping): $p(y|x_{in}) = p([\text{MASK}] = \mathcal{M}(y)|x_{\text{prompt=in+tmp}})$
Few-shot Prompting: Augment the prompt with input/output examples
Chain of Thought (CoT) Prompting: Eliciting the model to produce a step-by-step solution of a problem

5.2 Multimodal LMs

Multimodal Fusion: take both visual v & text w features; map to contextualized multimodal repr. $\tilde{v}, \tilde{w}$; Merged attention: concat features & feed into single transformer block; Co-attention: features fed into different tranformer blocks, then cross-attention between them
Multimodal Language pretraining: a) Masked language modeling: rand. mask input words; predict tokens based on surr. words & paired image; b) Image Text Matching

5.3 Instruction tuning

finetune LMs on collection / mixture of NLP tasks described via instructions → model can follow NEW instrusctions

5.4 Reinforcement Learning from Human Feedback (RLHF)

Instruct GPT: **1. Supervised Finetuning (SFT)**: human-written output for given prompt used to finetune model in supervised manner; **2. Collecting comparison data & Training a reward model**: prompt & several model outputs sampled; labeler ranks outputs; used to train REWARD model to predict human preferences; **3a Direct Preference Optimization (DPO)**; **3b. Learning from Human Feedback (RLHF)**: use reward model $R(s;\text{prompt})$ to obtain scores w/o human labeler; find best gen. model p_θ which maximizes: $\hat{\theta} = \text{argmax}_\theta \mathbb{E}_{\hat{s}\sim p_\theta}[R(s;\text{prompt})]$; GD update: $\nabla_\theta \mathbb{E}_{s\sim p_\theta}[R(s;p)] \approx \frac{1}{n}\sum_{i=1}^{n} R(s_i;p)\nabla_\theta \log p_\theta(s_i)$

5.5 RAG

LM's knowledge without RAG: hard to interpret, trust or modify
Parametric LMs: incorporate pretrained entity embeddings in token embeddings: align mentions in input text with entities in KB; inc. knowledge as add. memory layer in transformer
Vector Space Model: ex.: bag-of-words: docs & queries represented as |V|-dimensional vets; $cos(d_j, q) = \frac{d_j \cdot q}{\|d_j\|\|q\|}$; simplest repr.: matrix of word counts for each doc
tf-idf: improved repr. of BoW: (1) term freq. $\text{tf}_{t,d} = log(\text{count}(t,d)+1)$, (2) document freq df_t= #docs term t occurs in, (3) inverse doc freq. $\text{idf}_t = log\frac{\#\text{docs}}{\text{df}_t}$, (4) $\text{tf-idf}(t,d) = \text{tf}_{t,d} \cdot \text{idf}_t$; $\text{score}(q,d) = \sum_{t\in q} \frac{\text{tf-idf}(t,d)}{\sqrt{\sum_{d_i\in d} \text{tf-idf}^2(d_i,d)}}$; approximated: $\text{score}(q,d) = \sum_{t\in q} \frac{\text{tf-idf}(t,d)}{|d|}$; inverted index: for each query term list of docs containing it
BM-25: $\sum_{t\in q} \text{idf}_t \cdot \frac{\text{tf}_{t,d}}{k(1-b+b(\frac{|d|}{|d_{\text{avg}}|}))+\text{tf}_{t,d}}$, k: balance betw. tf & idf, b: importance of doc length norm.
BERT: $h_q = \text{BERT}_Q(q)[CLS], h_d = \text{BERT}_D(d)[CLS], \text{score}(d,q) = h_q \cdot h_d$
Dense passage retriever (DPR): 2 sep. encoders for question E_Q & text passages E_P; $\text{sim}(q,p) = E_Q(q)^T E_P(p)$
Incorporating retrieved knowledge: **(A)** Interpolate LM output probab. (kNN LM); augment LM with data store & interpol. $P_{LM}(x_{t+1}|x_{<t})$ with discrete distr. $P_D(x_{t+1}|x_{<t})$ estimated from data store; **(B)** Concat retrieved text with input (REALM); 1. knowledge retriever & knowledge-augmented encoder jointly pretrained on LM taks, 2. both are finetuned on specific task, e.g. Q&A; **(C)** incorp. retr. knowl. into tansformer layers by cross attention

6 SECURITY

6.1 Adversarial Examples

Adversarial Example: Input $\mathbf{x}+\delta$ s.t. $f(\mathbf{x}+\delta) \ne y$ and $\delta \in S$ for classifier $f(\mathbf{x})$, true label y, and pertub. set S.
Comment: Adv. examples as constrained optimization: maximize $\mathcal{L}(f_\theta(x+\delta), y)$, subject to $\delta \in S$, requires access to model weights
PGD attack: Starting from init. pert. $\delta \in S$
$\delta_{t+1} = \text{Clip}_{[-\varepsilon,\varepsilon]}((\delta_t + \alpha \cdot \text{sign}((\nabla_\mathbf{x}\mathcal{L}(f(\mathbf{x}+\delta_t), y)))))$
where $\text{Clip}_{[-\varepsilon,\varepsilon]}$ projects pert. back to S by clipping δ back into $[-\varepsilon,\varepsilon]$
Ex.: $\nabla p_{LM}(t_{i+1}|emb(e_1),\ldots,emb(e_m), emb(t_1),\ldots,emb(t_k); emb_{img}) = \nabla_{e_{img}} p_i \cdot \nabla_x emb_{img}(x)$
Attacking text-only LLMs: challenge: input space is discrete; simple pert. set: adv. example $\mathbf{x}' = \mathbf{x}(\text{orig. input}) + \delta(\text{adv. suffix})$; yields **Loss function** $\mathcal{L}_{\text{prefix}}(\mathbf{x}+\delta') = \mathcal{L}(y_1, \text{"Sure"}) + \cdots + \mathcal{L}(y_n, \text{"that"})$, $\mathcal{L}_{\text{prefix}}(\mathbf{x}+\delta') \approx \mathcal{L}_{\text{prefix}}(\mathbf{x}+\delta) + \nabla_{\mathbf{s}_i}\mathcal{L}'(s_1,\ldots,s_n)\cdot(s_i - s'_i)$, where s'_i embedding of ith token in δ'
Greedy Coordinate Gradient (GCG) Attack: computes $\nabla_{\mathbf{s}_i}\mathcal{L}'(s_1,\ldots,s_n)$ and picks k tokens that maximize loss approximation; select $B \le kn$ of all the candidate token substitutions across all positions; pick token replacement $\delta^{(i)}$ which maximizes $\mathcal{L}_{\text{prefix}}(\mathbf{x}+\delta^{(i)})$
Comment: GCG transfers betw. LLMs (white-box attack to black-box model)
Defenses: Perplexity filters, ML classifier, Adversarial training

6.2 Watermarking

bias LLM away from true $p(s_{i+1}|s_i)$
Hard Red-List WM Randomly partition vocab V into G & R list for each generated token based on prev. token as seed; model restricted to select tokens from G; z-test to eval. whether text has WM: $z = \frac{2(|s|_G - T/2)}{\sqrt{T}}$, T = #tokens, $|s|_G$ = #green list tokens used; large pos. z → likely WM
Soft Red-List WM $p_i = \begin{cases} \frac{e^{k_i-\delta}}{\sum_{j\in R} e^{k_j-\delta} + \sum_{j\in G} e^{k_j}} & \text{if } i \in R \\ \frac{e^{k_i}}{\sum_{j\in R} e^{k_j-\delta} + \sum_{j\in G} e^{k_j}} & \text{if } i \in G \end{cases}$, k : logits
Binom. distr.: $\mu = np, \sigma = \sqrt{n\cdot p\cdot q}$

6.3 Prompt injection

Defenses: train model against attacks; escape attached data;, 2nd LM to detect injection; instruction hierarchy; quarantined LLMs (SW wrapper injects actual data)

6.4 Data poisoning/Backdoors

corrupt moderate amount of training data; backdoored input-output pairs; mislabeled model completions; poisoned reward model

6.5 Model Stealing

Distillation: attacker tries to fit model $\hat{f}$ to data $(\mathbf{x}, f(\mathbf{x}))$
"Cryptanalysis": attacker queries model to obtain pred. on input feature vecs; if API returns confidence val $f_1(x)$, obtains LE $w \cdot x + b = \sigma^{-1}(f_1(x))$ for each query; querying d+1 random d-dim inputs, can solve d+1 unknown params
Attacking Transformers: A: recover hidden dim; 1. query model on n random propmts p_i to obtain logit vets $W \cdot g(p_i)$; 2. construct $Q = WH$ with each column being logit vec; 3. return rank of Q; B: recover embed proj. matrix: 1. SVD of $Q = U\Sigma V^T$; 2. $U\Sigma = WG$ for some G